

Malware Analysis Report

Reverse Engineering and Behavioral Analysis of a LockBit 4.0-Tagged Ransomware Sample

Disclaimer

This report was prepared for educational purposes, malware analysis, and defensive security research. The sample was analyzed inside an isolated virtual machine (VM) in a fully offline environment without an Internet connection. This report does not encourage the deployment, distribution, or unauthorized use of malware. The analysis focused on understanding the sample's structure, execution flow, and observable behavior. The information is intended to support malware research, threat detection, incident response, and defensive security practices.

1. Executive Summary

This report documents the analysis of a Windows 64-bit executable suspected to be associated with LockBit 4.0 ransomware. The sample was obtained from MalwareBazaar and examined inside a fully offline virtual machine to reduce the risk of exposing external systems to the malware.

The initial examination showed several characteristics commonly associated with packed malware, including a modified UPX signature, high file entropy, and a writable and executable memory section. These characteristics suggested that the executable contained a packed payload that required further investigation.

Dynamic analysis using x64dbg was used to observe the unpacking process and identify a possible Original Entry Point (OEP). Scylla was then used to reconstruct the Import Address Table (IAT) and create a memory dump for further examination in IDA Pro.

Static analysis identified a custom API resolution routine, a possible circular queue for file processing, and atomic operations that may support multithreaded execution. Signsrch also reported patterns associated with SHA-256, SHA-512, and a linear congruential generator (LCG).

Procmon was used to examine file system and registry activity. The recorded events showed a high volume of file operations and registry queries. Network-related observations from the supplied analysis indicated attempted connections to addresses in a local subnet, although successful connections and actual lateral movement were not established.

Overall, the investigation provided useful insight into the sample's structure, possible execution flow, and observable behavior. However, the exact identification of the sample as LockBit 4.0 and some of its reported capabilities still require independent verification

2. Sample Information

The following table contains the identification details of the analyzed sample.

Field	Information
SHA-256	43b7927b9f8669bcee4717a6902f404eb6f6ec05a4be2d59349400eaabf01ad5
MD5	f828044c91ac00affcd77b4ce680857
Sample source	MalwareBazaar
Malware type	Ransomware
Architecture	64-bit

2.1 Sample Classification Note

The sample was tagged as Expiro and LockBit 4.0 on MalwareBazaar. These labels should be treated as identification clues rather than definitive proof of the sample's origin or behavior.

The Expiro label is worth investigating because Expiro is also associated with a different malware family. A mismatch between a repository tag, antivirus detection names, and observed behavior can occur for several reasons, including inconsistent metadata, shared signatures, or incorrect classification.

For this reason, this report uses suspected LockBit 4.0 ransomware as the working classification. Confirming the exact family would require additional evidence, such as a verified sample reference, reliable family-specific signatures, or comparison with a known sample.

3. Analysis Objectives

The main objectives of this investigation were:

- Identify the sample's basic file properties and possible malware classification.
- Examine the executable's packing and memory-section characteristics.
- Observe the unpacking process and identify a possible OEP.
- Reconstruct the IAT to improve static analysis.
- Investigate API resolution, file-processing logic, and possible multithreading.
- Monitor file system, registry, and network-related activity.

4. Analysis Environment and Methodology

4.1 Laboratory Environment

The laboratory setup was intended to reduce the possibility of accidental communication with external systems during execution. No Internet connection was available to the VM during the experiment. The following tools were used during the investigation:

Tool	Purpose
Detect It Easy (DIE)	Identify file format, packer signatures, and executable properties
PEStudio	Examine PE headers, sections, imports, and suspicious file characteristics
x64dbg	Observe runtime behavior, memory changes, and execution flow
Scylla x64	Reconstruct the IAT and create a dumped executable
IDA Pro	Perform static disassembly and decompilation

Signsrch	Search for known byte patterns and algorithm-related signatures
Process Monitor (Procmon)	Monitor file system, registry, process, and network-related events

4.2 Analysis Workflow

The investigation followed this general workflow:

- a. Initial triage, examine the sample's hashes, file format, reported detection results, and repository tags.
- b. Packer identification, inspect the PE sections, entropy, and UPX-related characteristics.
- c. Dynamic unpacking, use x64dbg to observe the execution flow and locate a possible OEP.
- d. IAT reconstruction, use Scylla to identify imports and generate a memory dump.
- e. Static analysis, inspect the dumped executable in IDA Pro and investigate selected functions.
- f. Signature analysis, use Signsrch to identify patterns that may be associated with cryptographic or other algorithms.
- g. Behavioral monitoring, review Procmon logs for file, registry, and network-related activity.
- h. Correlation and reporting, compare the results from different tools and document the findings, limitations, and defensive recommendations.

5. Initial Triage and File Structure

5.1 File Reputation and Hashes

The sample's SHA-256 and MD5 hashes were recorded to establish its identity during the investigation. The reported VirusTotal result was 56 detections out of 69 engines. This indicates that the file was widely recognized as suspicious or malicious by the engines consulted at the time.

However, antivirus detection counts alone cannot confirm a specific malware family. Different engines may use different detection names, and generic detections do not always identify the same underlying threat.

5.2 PE Structure and UPX Detection

DIE and PESTudio reportedly identified the executable as a 64-bit Windows PE file with characteristics associated with a modified UPX packer.

The reported packer signature was UPX (3.91+) [modified]. The initial file size was approximately 617 KiB, and the reported entropy was 7.933. High entropy can indicate compressed or encrypted data, but it does not independently prove that the entire executable is encrypted. Combined with the packer signature and the observed memory behavior, it supports the possibility that the sample contains a packed payload.

The standard UPX unpacking command reportedly did not successfully unpack the sample. This is consistent with a modified or nonstandard packed executable, although the exact reason for the failure would require examination of the file's structure.

5.3 Memory Section Characteristics

The initial assessment identified the UPX0 and UPX1 sections, along with observations concerning .rdata. The UPX0 section was reported to have read, write, and execute permissions.

A writable and executable memory region can allow a loader to write unpacked instructions into memory and subsequently execute them. This behavior is relevant to unpacking analysis, but RWX permissions alone do not establish malicious intent because legitimate software can also use dynamically generated code.

The reported self-modifying behavior should therefore be understood in the context of the complete execution trace rather than as an isolated indicator.

6. Dynamic Unpacking and IAT Reconstruction

6.1 Runtime Memory Mapping

During debugging, the executable was reported to use the following image base 0x0000000014000000

The two main sections associated with the packed executable were reported at these virtual addresses:

Section	Reported virtual address
UPX0	0x0000000140001000
UPX1	0x0000000140119000

6.2 Identification of a Possible Original Entry Point

During dynamic debugging in x64dbg, a jump instruction was reportedly observed at 0x0000000140125182. The instruction was described as JMP 0x0000000140013A9F.

Execution subsequently reached 0x0000000140013A9F, where the disassembly reportedly showed a sequence of register-saving instructions, including PUSH R15, PUSH R14, and PUSH R13. This location was selected as the suspected Original Entry Point (OEP) because execution appeared to transition from the unpacking stub into a different code region.

The observation supports the identification of a possible OEP. Nevertheless, the instruction sequence alone is not sufficient to prove that the destination is the original program entry point. Additional validation, such as checking the memory state, restored code, and subsequent initialization routines, would strengthen this conclusion.

6.3 IAT Reconstruction Using Scylla

The reported IAT reconstruction was performed with Scylla x64 version 0.9.8. The recorded parameters were:

Parameter	Reported value
OEP	0x0000000140013A9F
IAT virtual address	0x0000000140126020

IAT size	0x100
Output file	..._dump_SCY.exe

The analysis reportedly identified Windows API functions such as ExitProcess, GetProcAddress, LoadLibraryA, and VirtualProtect.

The reconstructed executable was then used for further examination in IDA Pro. IAT reconstruction can improve the readability of a dumped executable by restoring import information. However, it does not automatically recover every original function name or guarantee that all imports and program structures have been reconstructed correctly.

6.4 Comparison of the Original File and Dump

The reported measurements were:

Metric	Original sample	Reconstructed dump
File size	617.00 KiB	736.50 KiB
Entropy	7.933	7.058

The increase in file size is consistent with the dump containing additional unpacked code or data. The reduction in entropy may indicate that the dump contains more structured or less-compressed content than the original sample. However, neither file size nor entropy alone proves that the unpacking process recovered the complete payload.

The dump should be treated as an analysis artifact derived from the original sample, not as a guaranteed byte-for-byte reconstruction of the original uncompressed executable.

7. Static Analysis in IDA Pro and Signsrch

7.1 Custom API Resolution

Static analysis reportedly identified a function labeled sub_14000D416, which appeared to perform custom API resolution. The function was described as using a numeric constant - 268106427

The reported behavior suggests that the sample may calculate hashes from API names and resolve Windows functions dynamically at runtime. This approach can make static analysis more difficult because the executable may not contain ordinary import references for every API it uses.

The analysis also reported indirect calls through a variable identified as v36. Indirect calls can reduce the number of cross-references that a disassembler automatically identifies, particularly when function pointers are resolved dynamically.

These observations are consistent with custom API resolution, but the exact hashing algorithm and the purpose of the numeric constant would need to be confirmed by tracing the function's inputs, calculations, and outputs.

7.2 Possible Multithreaded File-Processing Queue

A function labeled sub_140009E11 was reported to contain characteristics of a circular queue. The observations included:

- An array identified as qword_14001EED8.
- A modulo operation involving 0x400E.
- References to atomic operations such as `_InterlockedCompareExchange` and `_InterlockedIncrement`.

These characteristics may indicate a queue used to coordinate file-processing tasks between multiple threads. Atomic operations can help coordinate shared data and reduce race conditions when multiple threads access the same resource.

If the function is part of the ransomware's file-processing workflow, this design could support concurrent file handling. However, the observations do not independently establish that the queue performs encryption or that it improves encryption speed.

A stronger conclusion would require tracing the queue's producers and consumers and determining how queued items are connected to file access and cryptographic operations.

7.3 Signature Scanning Results

Signsrch reportedly identified signatures associated with:

- SHA-256
- SHA-512
- A linear congruential generator (LCG), with a reported constant of 0x343FD

These results suggest that the executable may contain code or data matching patterns associated with the listed algorithms. However, signature matches do not prove that the algorithms are actively used to encrypt files. They may originate from embedded libraries, unused code, or coincidental byte patterns.

Further analysis would be needed to establish whether these routines are called during file processing and how their outputs are used.

8. Behavioral Analysis Using Procmon

8.1 Network-Related Activity

The supplied analysis reported sequential connection attempts involving addresses within the local subnet 192.168.37.0 – 192.168.37.254

The activity was associated with TCP port 445, commonly used by Microsoft-DS/SMB services. This behavior may indicate an attempt to discover or communicate with SMB services on nearby systems. Such activity can be relevant when investigating possible network discovery or lateral movement.

However, the available observations do not establish that the sample successfully connected to other computers, accessed network shares, exploited SMB vulnerabilities, or spread to other systems.

Because the VM was fully offline, it is particularly important to distinguish recorded connection attempts from successful network communication. The source of the events, their exact Procmon operation types, and the laboratory's virtual network configuration should be checked before drawing conclusions about actual connectivity.

The reported reconstructed dump produced zero network-related events in the compared capture. This difference may indicate that the dump followed a different execution path or that the observed activity depended on the original sample's runtime state. It does not, by itself, prove that network functionality was removed from the dump.

8.2 Registry Activity

Procmon observations reportedly included reads involving language-related settings, including:

- LanguageConfiguration
- PreferredUILanguages

These queries may be related to locale detection or general system configuration checks. Some ransomware families use locale information when deciding whether to continue execution. However, the presence of registry reads does not prove that this sample contains a country-based exclusion list or terminates when it detects a particular region.

To confirm such behavior, the registry query results would need to be traced to the relevant conditional branches and their resulting execution paths.

The analysis also reported no observed modifications to common Windows Run keys. This suggests that no such persistence modifications were observed during the monitored execution. It does not prove that the sample has no persistence capabilities, since other mechanisms or execution paths may exist.

8.3 File System Activity

The original sample reportedly generated approximately 194,548 Procmon events during the observation period. The supplied event summary included:

Operation	Reported count
Opens	69,398

Reads	12,584
Writes	8,340
Total events	194,548

The reported read volume was 396,702,789 bytes, while the write volume was 234,219,644 bytes. These figures indicate substantial file system activity during the captured execution.

However, Procmon event counts and byte totals must be interpreted carefully. Repeated operations, file metadata queries, temporary files, failed operations, and other system activity may contribute to the totals.

The difference between reported read and write volumes cannot independently establish the number of encrypted files, the encryption ratio, or the amount of data successfully encrypted.

To confirm ransomware activity, the investigation should correlate file operations with actual file-content changes, file renaming, extension changes, ransom note creation, or other family-specific behavior.

9. Indicators of Compromise (IoCs)

The following table summarizes the identifiers and behavioral indicators reported during the investigation.

Indicator	Value or observation	Confidence
SHA-256	43b7927b9f8669bcee4717a6902f404eb6f6ec05a4be2d59349400eaabf01ad5	High for sample identification, assuming the hash was calculated correctly
MD5	f828044c91ac00affcd77b4ce680857	High for sample identification, assuming the hash was calculated correctly
File type	Windows PE32+, 64-bit	Reported file property

Packer indicator	Modified UPX signature	Reported static-analysis result
Memory indicator	UPX0 section reported with RWX permissions	Reported file/runtime characteristic
Runtime indicator	Possible transition from unpacking stub to payload at 0x0000000140013A9F	Requires OEP validation
API resolution	Possible custom API hashing and indirect calls	Requires algorithm and call-flow validation
Network indicator	Reported connection attempts involving 192.168.37.0/24 and port 445	Requires event-level validation
File activity	High volume of file opens, reads, and writes	Reported behavioral observation
Registry activity	Language and locale-related queries	Reported behavioral observation
Persistence	No Run-key modifications observed	Limited to the captured execution

10. Limitations of the Investigation

Several limitations should be considered when interpreting the results.

10.1 Malware Family Attribution

The MalwareBazaar tags include both Expiro and LockBit 4.0. These labels are not sufficient to establish a definitive family attribution. Additional evidence is required to confirm whether the sample is a genuine LockBit variant, a different malware family, or a sample with inconsistent repository metadata.

10.2 Offline Execution

The VM was fully offline, which reduced the risk of external communication during testing. However, this setup also limits the ability to observe behaviors that depend on reachable network services, command-and-control infrastructure, remote resources, or other computers. Recorded connection attempts do not necessarily represent successful communication.

10.3 Dynamic Analysis Coverage

The observed behavior represents the execution paths reached during the experiment. Malware may behave differently depending on its execution environment, user privileges, system configuration, available files, locale, and other conditions. The absence of an observed behavior does not necessarily mean that the capability is absent.

10.4 Unpacking and Static Analysis

The reported OEP, IAT, and reconstructed dump provide useful starting points for further investigation. Nevertheless, the integrity and completeness of the dump have not been independently established from the information available in this report.

10.5 Cryptographic and Encryption Findings

Signature matches, multithreading indicators, and file system activity provide clues about possible ransomware functionality. They do not independently establish the exact encryption algorithm, key management design, encryption success rate, or file-processing logic. These conclusions require additional evidence from the relevant code paths and controlled file-content comparisons.

11. Recommendations

The following recommendations are intended for security teams, malware researchers, and system administrators.

11.1 Improve Malware Identification

Before sharing the report, verify the SHA-256 and MD5 hashes against the original sample to ensure that the sample has been correctly identified. Review the MalwareBazaar record and its associated metadata to investigate the classification mismatch between Expiro and LockBit 4.0. Compare the sample with trusted family-specific references before making a final decision about its malware family. The original sample and the unpacked dump should also be stored separately, with their hashes and sources documented to maintain file integrity and traceability.

11.2 Improve Behavioral Validation

Behavioral analysis should be based on detailed event records rather than summary counts alone. Review the Procmon logs to determine whether network-related events represent attempted connections, successful connections, or failed operations. The original executable and the unpacked dump should also be compared under equivalent laboratory conditions to identify differences in their behavior. Suspected locale checks should be validated by examining the relevant code branches and their execution results.

11.3 Strengthen Endpoint Protection

Organizations should maintain regular backups of critical data and ensure that the backups are tested and stored offline or in otherwise isolated locations. Endpoint Detection and Response (EDR) solutions should be used to monitor suspicious file modifications, unusual process behavior, and unexpected executable memory permissions. Unnecessary SMB access between workstations and network segments should be restricted, while unusual connections to TCP port 445 should be monitored and investigated. Organizations should also apply the principle of least privilege by limiting administrative access to authorized users and keeping operating systems and security software up to date.

11.4 Improve Detection Engineering

The findings from this investigation can be used as a starting point for developing detection mechanisms based on multiple suspicious behaviors rather than relying on a single indicator. Potential detection opportunities include unusual executable memory activity associated with unpacking, suspicious runtime API resolution combined with abnormal process behavior, and high volumes of file modifications within a short period. Unexpected SMB connection attempts to numerous internal addresses may also deserve investigation, especially when they occur alongside unusual file access, ransom note creation, or mass file renaming.

12. Conclusion

This investigation examined a Windows 64-bit executable obtained from MalwareBazaar that was reported as a suspected LockBit 4.0 ransomware sample.

The analysis identified characteristics associated with a modified UPX packer, including high entropy and a reported writable and executable memory section. Dynamic debugging provided a candidate OEP, and Scylla was used to reconstruct the IAT and generate a dump for further static analysis.

The static investigation reported possible custom API resolution, a circular queue, atomic operations, and algorithm-related signatures. Procmon observations also showed substantial file system activity, locale-related registry queries, and reported attempts to communicate with addresses on a local subnet.

Together, these findings provide a useful foundation for understanding the sample's structure and possible behavior. However, the available evidence does not independently confirm every reported capability or establish a definitive LockBit 4.0 attribution. In particular, the Expiro repository tag, the exact encryption workflow, and the success of the reported network activity require further validation.